

PERTEMUAN 10

PEMROGRAMAN BERORIENTASI OBJEK

10.1 Pengertian Constructor

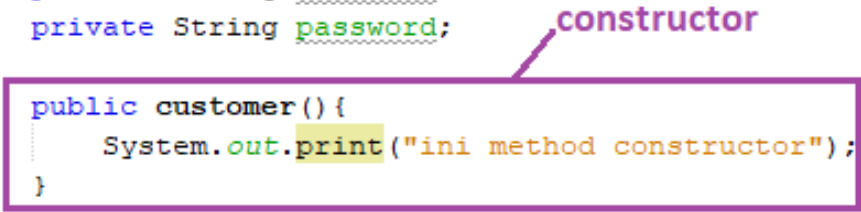
Constructor merupakan method khusus yang akan dieksekusi saat pembuatan objek (*instance*). Biasanya method ini digunakan untuk inisialisasi / mempersiapkan data objek.

Contoh Constructor:

```
package konstruktor;

public class customer {
    private String username;
    private String password;

    public customer() {
        System.out.print("ini method constructor");
    }
}
```



Cara membuat constructor adalah dengan menuliskan nama method constructor sama seperti nama class. Pastikan memberikan modifier public kepada Constructor, karena ia akan dieksekusi saat pembuatan objek (*instance*).

>> customer.java <<

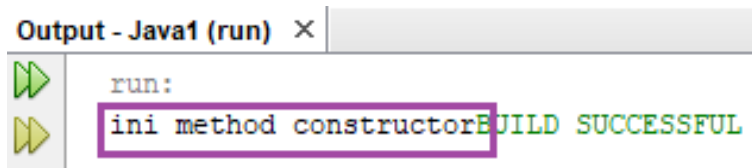
```
package konstruktor;

public class customer {
    private String username;
    private String password;

    public customer() {
        System.out.print("ini method constructor");
    }
}

class contohconstructor{
    public static void main(String[] args){
        customer galuh = new customer();
    }
}
```

Output programnya:



10.1 Constructor dengan Parameter

Constructor biasa digunakan untuk *initialize* (menyiapkan) data untuk class. Untuk melakukan ini, buat parameter sebagai inputan untuk constructor. Contoh:

```
public class customer {  
    public String username;  
    public String password;  
  
    public customer(String username, String password){  
        this.username = username;  
        this.password = password;  
    }  
}
```

Pada kode class customer di atas, dapat ditambahkan parameter username dan password kedalam constructor. Jadi saat membuat objek, harus menambahkan nilai parameter. Contoh:

```
customer galuh = new customer("Galuh Saputri", "12345abc");
```

Buka kembali **customer.java**, modifikasi seperti kodingan berikut:

```

package konstruktor;

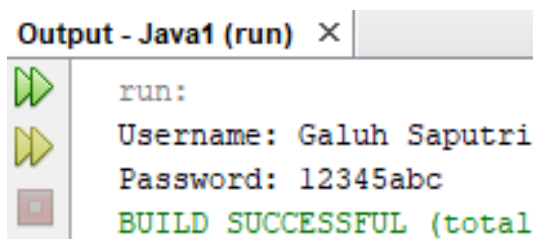
public class customer {
    public String username;
    public String password;

    public customer(String username, String password){
        this.username = username;
        this.password = password;
    }
}

class contohconstructor{
    public static void main(String[] args) {
        customer galuh = new customer("Galuh Saputri", "12345abc");
        System.out.println("Username: " + galuh.username);
        System.out.println("Password: " + galuh.password);
    }
}

```

Output programnya:



```

Output - Java1 (run) X
run:
Username: Galuh Saputri
Password: 12345abc
BUILD SUCCESSFUL (total

```

10.2 Method Setter dan Getter

Method setter dan getter merupakan dua method yang bertugas mengambil dan mengisi data ke dalam objek. Pada Pemrograman Berorientasi Objek ada istilah encapsulation (pembungkusan) dimana data dibungkus dengan modifier private agar tidak dapat diakses secara langsung dari luar class, dengan method setter dan getter maka data tersebut dapat diakses.

Fungsi method Setter dan Getter:

1. Meningkatkan keamanan data
2. Lebih mudah dalam mengontrol atribut dan method
3. Class dapat dibuat menjadi *read only* dan *write only*
4. Fleksibel

10.3 Membuat Method Setter dan Getter

Membuat method setter dan getter sama seperti membuat method yang lain, contoh:

```
class contohsettergetter {  
    private String nama;  
    private int umur;  
  
    // ini method setter  
    public void setName(String nama){  
        this.nama = nama;  
    }  
    public void setUmur(int umur){  
        this.umur = umur;  
    }  
    // ini method getter  
    public String getName(){  
        return this.nama;  
    }  
    public int getUmur(){  
        return this.umur;  
    }  
}
```

Method setter dan getter harus menggunakan modifier public, karena akan diakses dari luar class. Perbedaan method setter dengan getter terletak pada nilai kembalian, parameter, dan isi method. Method setter tidak memiliki nilai kembalian (void) karena tugasnya hanya mengisi data ke dalam atribut. Sedangkan method getter memiliki nilai kembalian (return) sesuai dengan tipe data yang akan dipakai. Nama pada method setter dan getter harus diawali dengan **set untuk setter** dan **get untuk getter**.

```
// ini method setter
public void setName(String nama) {
    this.nama = nama;
}

// ini method getter
public String getName() {
    return this.nama;
}
```

10.3 Method Setter dan Getter Menggunakan Fitur Generator

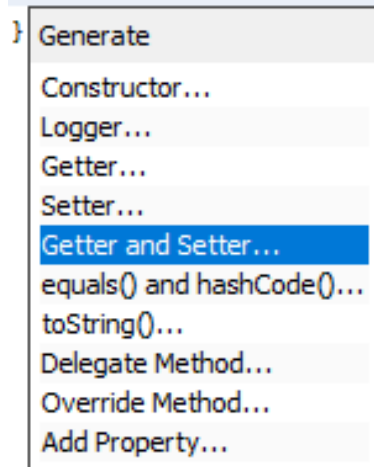
Jika banyak data dengan lebih dari 10 atribut yang membutuhkan method setter dan getter, dapat menggunakan fitur generator di Netbeans.

1. Buat java class dengan nama **Manusia.java**
2. Buat atribut data yang akan dibuat method setter dan getter

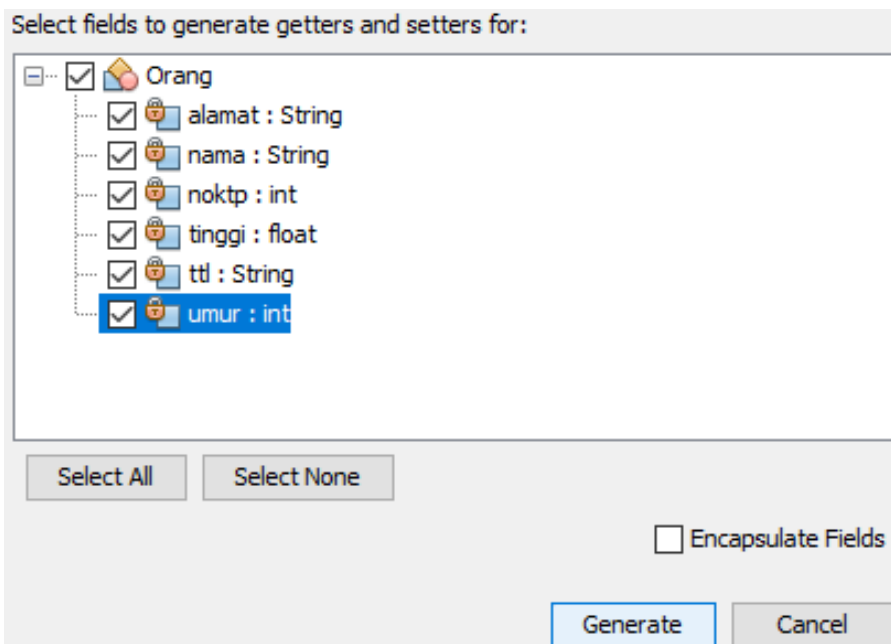
```
class Orang {
    private String nama;
    private int noktp;
    private String alamat;
    private String ttl;
    private int umur;
    private float tinggi;
}
```

3. Setelah itu klik kanan, lalu pilih **Insert Code**, akan tampil menu seperti dibawah:

```
class Orang {
    private String nama;
    private int noktp;
    private String alamat;
    private String ttl;
    private int umur;
    private float tinggi;
}
```



4. Pilih **Setter and Getter**, ceklis atribut yang akan dibuat method setter dan getter, kemudian klik Generate.



5. Akan tampil setter dan getter pada class Manusia secara otomatis

```
class Orang {
    private String nama;
    private int noktp;
    private String alamat;
    private String ttl;
    private int umur;
    private float tinggi;

    public String getNama() {
        return nama;
    }
    public void setNama(String nama) {
        this.nama = nama;
    }
    public int getNoktp() {
        return noktp;
    }
    public void setNoktp(int noktp) {
        this.noktp = noktp;
    }

    public String getAlamat() {
        return alamat;
    }
    public void setAlamat(String alamat) {
        this.alamat = alamat;
    }
    public String getTtl() {
        return ttl;
    }
    public void setTtl(String ttl) {
        this.ttl = ttl;
    }
    public int getUmur() {
        return umur;
    }
    public void setUmur(int umur) {
        this.umur = umur;
    }
    public float getTinggi() {
        return tinggi;
    }
    public void setTinggi(float tinggi) {
        this.tinggi = tinggi;
    }
}
```

6. Lanjutkan source code untuk mengisi data dengan membuat class baru **dataManusia.java**.

```
package LatihanJava;

public class DataManusia {
    public static void main (String[] args){

        Manusia manusia = new Manusia();

        manusia.setNama("Galuh Saputri");
        manusia.setNoktp(1234567);
        manusia.setAlamat("Ciseeng");
        manusia.setTtl("Jakarta, 1 Juli 1996");
        manusia.setUmur(29);
        manusia.setTinggi(155);

        System.out.println("Nama : "+manusia.getNama());
        System.out.println("No. KTP : "+manusia.getNoktp());
        System.out.println("Alamat : "+manusia.getAlamat());
        System.out.println("Tempat Tanggal Lahir : "+manusia.getTtl());
        System.out.println("Umur : "+manusia.getUmur());
        System.out.println("Tinggi : "+manusia.getTinggi());
    }
}
```